

Genealogiq — Engagement Summary

Hardening and consolidation of the Genealogiq platform — security, architecture, and maintainability.

Date: June 2026 **Scope:** Three Next.js 16 / React 19 / TypeScript applications sharing one Neon PostgreSQL database, consolidated into a single pnpm + Turborepo monorepo and hardened end to end.

1. Context

Genealogiq runs as three separate applications against one shared database:

App	Role	Domain
BMS	Back-office / admin	bms.genealogiq.app
SEQ (Sequoia)	Funeral-home tenant portal (multi-tenant B2B)	sequoia.rip
APP	Consumer-facing B2C product	genealogiq.app

At the start of the engagement the three lived as independent repositories with duplicated code, three drifting copies of the Prisma schema, no shared tests, no CI gates, and a set of security findings affecting live data. The goal was to make the platform **safe, consistent, and maintainable** without disrupting the three independent deployments or their domains.

The work was sequenced deliberately from lowest to highest risk: fix the urgent security holes first, then build the safety net (CI + tests), then consolidate the code, and only then touch the shared database schema. Every change shipped through its own branch, pull request, and merge, with the database changes applied behind Neon snapshots.

2. What was done

2.1 Security fixes (first priority)

A security audit surfaced several classes of exploitable issues, all of which were fixed and verified:

- **Broken access control / IDOR (Insecure Direct Object Reference).** Multiple endpoints trusted client-supplied identifiers without verifying ownership. A user could moderate tributes, read or edit family-tree data, or reach another tenant's guardian records that did not belong to them. Each was fixed by enforcing ownership and tenant scoping at the data-access layer (findings C1–C4).
- **Authorization logic gaps.** A profile-management check accepted a status it should not have, widening who could manage a profile; sale-creation accepted unvalidated monetary values. Both were tightened (A2, A3).
- **Upload hardening.** File-upload routes across all three apps were hardened against abuse (A1).
- **Stripe webhook integrity.** Payment fulfillment was made atomic so a half-processed webhook can no longer leave an order in an inconsistent state, with proper error handling and schema validation on the payload (P1).
- **Token hashing.** Email-verification and password-reset tokens are now stored hashed rather than in plaintext, so a database leak no longer hands an attacker working tokens (P1-10).
- **Data exposure.** Sensitive fields (national ID, phone, notes, address) are now redacted or masked according to the caller's privilege, instead of being returned wholesale by the API (M1–M3, B1).

2.2 Consolidation into a monorepo

The three repositories were merged into a single pnpm + Turborepo monorepo (`apps/bms`, `apps/seq`, `apps/app`), **preserving the full git history** of each app (414 commits total). The three Vercel deployments and their domains were preserved by pointing each project at the same repo with a different root directory.

Shared code was then extracted into six internal packages, eliminating the duplication and drift:

Package	What it owns
<code>@genealogiq/core</code>	Cross-cutting primitives (e.g. token hashing, the <code>Result<T></code> type)
<code>@genealogiq/ui</code>	The shared shadcn/ui component library (unified from BMS + SEQ)
<code>@genealogiq/db</code>	One canonical Prisma schema — the single source of truth for the database
<code>@genealogiq/auth</code>	NextAuth v5 configuration, the data-access layer, session model, sign-in flow, and account-lockout policy
<code>@genealogiq/email</code>	Transactional email transport and templates
<code>@genealogiq/services</code>	Remaining shared services (codes, rate-limiting, Stripe)

The single canonical schema (Phase 4) was the highest-risk step and was done last. Rather than reconcile three

hand-edited schemas by guesswork, the live database was treated as the source of truth: the schema was derived directly from production via `prisma db pull`, and all three apps were re-pointed at it.

2.3 A foundation that leads, instead of adapting (Phase 5)

The guiding principle for the shared auth/data foundation was deliberate: **define the right way once and move all three apps to it**, rather than copying whatever each app happened to do. The original code contained genuine mistakes and accidental variation; the consolidation distinguished *essential* differences (cookie names, which identity table each app uses, tenant scoping) from *accidental* ones (everything else), and converged the accidental ones.

Concretely, this produced a factory-based foundation: one implementation of the edge auth config, the auth handler, the data-access layer, the sign-in action, and the credential-authorization flow, each parameterized per app. Account lockout (5 failed attempts, 15-minute window) is now enforced **uniformly** across all three apps — previously only one app enforced it. Email templates converged to a single English set (only one app had diverged into Portuguese).

2.4 Safety net and hardening

- **CI gates.** A monorepo CI pipeline runs on every push and pull request: Prisma client generation, type-checking, the unit-test suite, a single-schema parity guard, and linting — all blocking. A dependency audit runs informationally.
 - **Tests.** A Vitest suite was stood up across the apps and packages, with targeted coverage of the security-critical paths: tenancy/IDOR enforcement, the account-lockout state machine, the QR-activation race condition, and Stripe checkout fulfillment.
 - **Database indexes.** Foreign-key and hot-path indexes were added (PostgreSQL does not auto-index foreign keys), focused on the back-office query patterns.
 - **Build correctness.** Pages that query the database without a per-request signal were marked `force-dynamic` so the build no longer reaches the database during static prerendering.
 - **Supply chain.** Dependabot and a CI dependency audit were wired in. After triage, Dependabot was tuned to a **security-and-minor-only** posture: it still opens pull requests for any vulnerable dependency, but no longer generates routine major-version noise.
 - **Migration baseline.** The Prisma migration history (which had diverged from the live database during the schema consolidation) was reconciled and documented, and the last two long-standing schema deviations were closed so the schema and the live database now match exactly.
-

3. State of the platform now

The three apps now share **one** database schema, **one** authentication and data-access foundation, **one** component library, and **one** CI pipeline — with their deployments, domains, and per-app secrets kept independent. The known security findings are fixed and covered by tests. CI blocks regressions on type errors, test failures, schema drift, and lint violations. The database schema and the live database are in exact agreement, with a clean, documented migration baseline.

In short: the platform moved from three drifting codebases with open security issues to a single, tested, gated monorepo with a deliberately-designed shared foundation.

4. Recommendations and open items

None of the following is urgent; they are the natural next increments.

- **Dependency majors.** TypeScript 6, ESLint 10, and Vitest 4 are available as major upgrades. There is no rush — the apps run fine today — but they should be done deliberately, one at a time on their own branches with the full test suite, rather than batched. Dependabot will no longer nag about them; pick them up when convenient.
 - **Confirm security updates are enabled.** In the GitHub repository settings (Settings → Code security), verify that "Dependabot security updates" is on. That is the switch that keeps security pull requests flowing under the new quieter configuration.
 - **Lint/type debt.** A handful of low-value lint warnings remain (unused variables, idle `eslint-disable` comments). These can be cleaned incrementally and are not blocking.
 - **Operational discipline to preserve.** Database DDL should continue to be applied behind a Neon snapshot, and the migration history should stay reconciled (the runbook in `MIGRATION.md` documents the baseline and why the orphan history rows must not be hand-deleted).
-

5. Appendix — reference

- **Architecture decision:** `ADR-0001-monorepo-consolidation.md`
- **Migration runbook & database baseline:** `MIGRATION.md`
- **Apps:** `apps/{bms, seq, app}`
- **Shared packages:** `packages/{auth, core, db, email, services, ui}`
- **CI pipeline:** `.github/workflows/ci.yml`
- **Dependency automation:** `.github/dependabot.yml`